2026 04/01 Narrative evermemos review和计划

1. 背景

LoCoMo benchmark测试（长期对话记忆）暴露了NarraNexus在长对话场景中的系统性问题：

- Narrative滚雪球 — 200+对话归入一个narrative，信息密度过高导致检索失败
- 连续性检测器过度聚合 — LLM持续判定"属于同一话题"，narrative无限膨胀
- EverMemOS检索准确性不明确 — 检索到的信息是否真正进入了最终回答
- 时间信息丢失 — LoCoMo强调时间敏感问答，但系统缺乏时间维度

同时结合我们日常使用中如narrative创建频繁问题， 进行review

---

2. 系统架构概览

2.1 核心概念

目前状态

Evermemos 也会通过episode retrieval辅助narrative选择

组件	定位	说明
Narrative	话题级= (Topic-level routing container)	持有元数据（name, summary, keywords, embedding, actors），链接module instances。**不存储对话内容本身**，是索引而非数据库
ChatModule Memory	原始记忆层 (Raw memory layer)	按(user_id, narrative_id)的完整对话记录，即时写入。前端Chat UI数据源。短期记忆和continuity path长期记忆的唯一来源

EverMemOS	语义记忆层 (Semantic memory layer)	将原始消息处理成Episode（边界检测→摘要→向量化），提供RRF检索（BM25+向量）。按group_id(=narrative_id)组织
Native VectorStore	Narrative级向量缓存	内存中routing_embedding余弦搜索。EverMemOS不可用时的后备。**非EverMemOS的替代品**——只做Narrative级匹配

2.2 数据存储

Store	位置	主键	内容	写入时机
Narrative	MySQL	narrative_id	name, summary, keywords, routing_embedding, actors, event_ids	每轮更新 (Step 4.4)
ChatModule	MySQL	instance_id = (user_id, narrative_id)	原始消息对 [{role, content}]	每轮 (Step 5 hook)
EverMemOS	MongoDB+ES+Milvus	group_id(=narrative_id) + user_id	Episodes: 语义摘要+向量	每轮 (Step 5 hook, async)
Session	JSON文件	(agent_id, user_id)	current_narrative_id, last_query/response, 超时600s	Narrative选择后

2.3 EverMemOS内部层级

Code block

```
原始消息 → MemCell（边界检测的对话chunk，多条消息）
                → Episode（MemCell的LLM摘要版本，含summary + 全文 + 向量）
                → Cluster（同一group内按语义+时间(7天)聚合的MemCell集合）
                → Group (= narrative_id)
```

3. 额外的cognitive input

3.1 四层记忆模型 — Roger Schank的Dynamic Memory (1982)

Schank层级	定义	对应我们系统	覆盖程度
Event Memory (具体事件记忆)	特定经历的记忆	ChatModule原始消息 + EverMemOS Episode	较好覆盖
Generalized Event Memory (泛化事件记忆)	从多个事件中抽象出的共同特征	EverMemOS Cluster (按语义聚合的episodes)	部分覆盖（Cluster存在但不参与检索）
Situational Memory (情境记忆)	上下文和规则	**缺失** — 无情境层表示	未覆盖
Intentional Memory (意图记忆)	目标和计划	**缺失** — Narrative无goal/intention字段	未覆盖

Schank还提出了**TOPs (Thematic Organization Points)** — 跨领域关联结构，解释为什么听到贸易谈判会联想到象棋。我们系统当前**无跨narrative关联检索机制**。

3.2 Conway自传记忆模型 — Self-Memory System (SMS)

Code block

```
Conway层级:     Life Story → Lifetime Periods → General Events → Event-Specific Knowledge
我们的系统:     (缺失)     →  Narrative       → Cluster(未用) → Episode/ChatModule
```

Conway模型有3层检索，我们只用了2层（Narrative → Episode）。 支持top-down, bottom up 检索后者访问某个层级信息

3.3 记忆检索理论

检索模型	核心机制
ACT-R (Anderson, 1993)	Activation = base-level(recency×frequency) + spreading activation(context)
SAM (Raaijmakers & Shiffrin, 1981)	多线索**乘积**组合，不是加和
Generative Agents (Park et al., 2023)	Score = recency × importance × relevance

Code block

```
Score = α(semantic_similarity) × β(recency_decay) × γ(importance) × δ(access_frequency)
```

---

4. 核心问题分析

主要分为两部分, evermemos与当前narrative, chatmodule memory 配合以及Narrative本身调整

Part A: EverMemOS与当前系统的耦合问题

核心判断：EverMemOS记忆与当前narrative耦合严重，两者间的对应关系破坏了EverMemOS memory summarization的独立能力。

A1. Scope重叠 — Narrative和Episode/Cluster概念边界模糊

层级	Scope	时间跨度
Episode	连续消息的语义chunk	分钟级

Cluster	按语义+时间聚合的episodes	天级 (7天max)
Narrative	应该 > Cluster，一个持续的项目/目标/话题	周/月级

问题： 当前一个query就能创建narrative，scope可以和episode一样小。Narrative和episode在概念上重叠。以我们一开始创建narrative就定了scope的方式，可能并不合适。要么LLM创建很好，要么得扩大scope到lifetime级别。

A2. 检索逻辑违和 — Middle-out selection

Code block

```
理想：Top-down（大到小：life plan → period → general event → specific event）
      或 Bottom-up（相关记忆聚合到话题）

现实：Middle-out（episode得分 → 反推narrative → 加载ChatModule历史）
      从中间层选到高层，然后直接加载底层，跳过了中间层
```

对比： EverMemOS episode + short-term memory vs Narrative额外提供了什么？

- 有：actor/participant信息，related job（作用稀释），high-level summary metadata，orchestration（模块实例）
- 没有：更多memory content，更好的检索信号

Narrative在检索中的角色不清晰

A3. ChatModule Memory vs EverMemOS Episodes — 定位混淆

现状： 两者在long-term memory功能上互为fallback，而非分层配合。

- Continuity path（最常见路径）：仅ChatModule DB最近40条消息，EverMemOS完全不加载
- Non-continuous path：EverMemOS episode_contents作为长期记忆
- 为什么continuous来源只有一个narrative，而non-continuous时有evermemos + 多个narrative?

ChatModule不可替代的作用：

- Short-term memory（其他narrative的15条最近消息）— 唯一来源
- Continuity path长期记忆 — EverMemOS未加载时的唯一来源
- 前端Chat UI — 唯一数据源
- 即时可用 — 无处理延迟

原则问题： EverMemOS是否应作为核心组件长期存在

Part B: Narrative自身的设计问题

B1. Narrative的边界和自整理能力

现状： Narrative有insert, update, 但没有reconstruct。搭配近乎无上限的summary。

共识： 参考Claude dream, 在不繁忙时间进行cross narrative, 结构性自整理

B2. Narrative与Social Network联动

问题： Participant信息在LLM判断中仅使用topic_hint[:50/100]，比较单薄。和Social Network有丰富的entity信息（role, description, contact）怎么搭配。

共识： 目前narrative依旧储存actor, participants等信息，social network agent自行调用，在larkcase中观测效果

5.当前的原则

P1: Narrative, evermemos完全解耦

Evermemos 只在系统中负责提供对应量级数据，为memory retrieval，每个agent划分，充分利用它内部聚类能力

Narrative update以及选择完全由自身信息决定

P2: Context来源一致

Job trigger, continuous, non-continuous应使用相同的分层context：

- Top level: Narrative metadata
- 中间层1: Cluster (optional, from EverMemOS)
- 中间层2: Episode (from EverMemOS)
- 底层: Short-term memory + detailed events (from ChatModule)

P3: 长度管理交到内部，不在调用处截断

每个数据源在内部提供有界输出。Narrative提供自行compacted的summary；Episode提供有界的summaries。不在LLM调用处截断。

P4: Narrative内部自整理机制

每天跨narrative进行review, 信息整合，divide, merge, 增减量等操作。让narrative每一个不至于过于臃肿或者单薄。在narrative中如果出现信息不continuous情况重新调整

---

6. 行动项

立即修复 (Bug)

项目	描述
EverMemOS agent_id隔离	添加agent_id为一等可过滤字段（EverMemOS约25文件）。已在ACTION_PLAN中完成scope
移除LLM调用处截断	数据源应自行限制context量。

项目	描述
Narrative summary compaction	在narrative updater中添加compaction机制。超过阈值时LLM压缩。移除description字段，仅保留current_summary
后台整理机制	白天增量更新，夜间全局整理：narrative合并拆解、动态更新memory。结合agent可演化机制
Narrative centroid retrieval	新增narrative_centroids表，增量event embedding平均。额外的ranking signal。特性分支已实现
Evermemos, narrative完全解耦	Evermemos 做信息补充，narrative负责提供高层信息以及和social network等合作

Hongyi Gu hongyi.gu 2026-04-07 11:29 UTC